

2.5 Loop Design

Sam is upset because he is trying to find a copy of the ASCII Table containing both hexadecimal (base 16) values and decimal (base 10) values. None of his favorite C++ web sites have the right table and Google is turning out to be useless. Rather than stooping to ask the professor, Sam decides to write his own code to display the table. But how to start? What will the main loop look like? (For an example solution of this problem, please see `/home/cs124/examples/2-5-asciitable.cpp`).

Objectives

By the end of this class, you will be able to:

- Recognize the three main types of loops.
- Use a loop to solve a complex problem.

Prerequisites

Before reading this section, please make sure you are able to:

- Demonstrate the correct syntax for a WHILE, DO-WHILE, and FOR loop (Chapter 2.3).
- Create a loop to solve a simple problem (Chapter 2.3).

Three Types of Loops

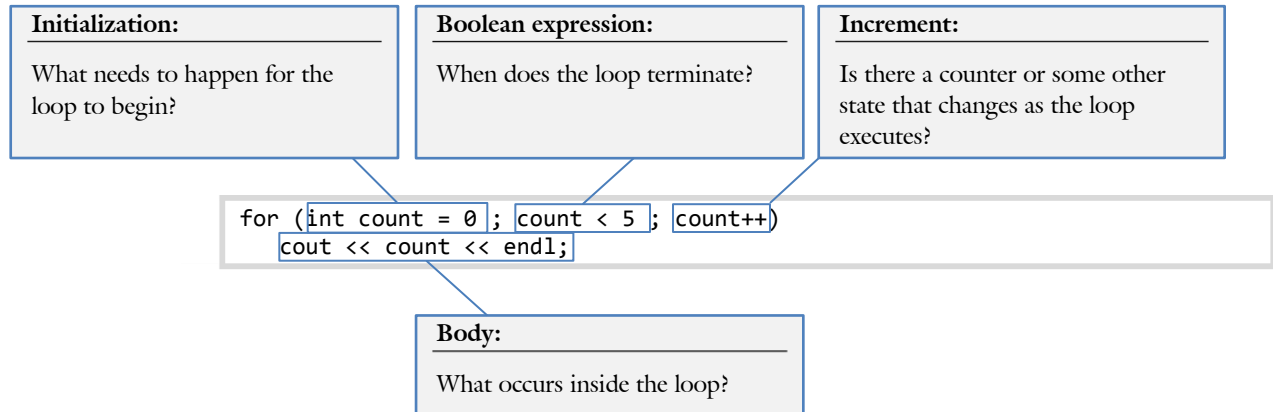
It takes a bit of skill to think in terms of loops. The purpose of this chapter is to give you design practice working with this difficult construct. Almost all looping problems can be broken into one of three categories: counter-controlled, event-controlled, and sentinel controlled. It is usually a good idea to identify which of the three types your problem calls for and design accordingly.

- **Counter Controlled:** Keep iterating a fixed number of times. The test is typically a single integer that changes with each iteration.
- **Event Controlled:** Keep iterating until a given event occurs. The number of iterations is typically unknown until the program runs.
- **Sentinel Controlled:** Keep iterating until a marker (called a sentinel) indicates the loop is done. The test is a given variable (typically a bool) that is set by any one of a number of events.

Counter-Controlled

A counter-controlled loop is a loop executing a fixed number of times. Typically that number is known before the loop starts execution. If, for example, I were to determine the number of students who had completed the homework assignment, a counter-controlled loop would be the right tool for the job.

In almost all circumstances a `for` statement is used for counter-controlled loops because `for` statements are designed for counting. Observe how the four parts to a `FOR` loop correspond to the four parts of the typical counting problem:



Counter-controlled loops are readily identified by the presence of a single variable that moves through a range of values. In other words, counter-controlled loops do not exclusively increment by one: they might increment by 10 or powers of 3.

When designing with a counter-controlled loop, it is usually helpful to answer the following four questions:

- **How does the loop start?** In other words, what is the beginning of the range of values? This corresponds to the initialization part of the `FOR` loop.
- **How does the loop end?** In other words, by what condition do you know that you have yet to reach the end of the range of values? This corresponds to the Test or controlling Boolean expression part of the `FOR` loop.
- **What do you count by?** In other words, as you move through the range of values, how does the variable change? This corresponds to the Update part of the `FOR` loop.
- **What happens each iteration?** In other words, is some action taken at each step of the counting? Frequently no action is taken so this step can be skipped. If there is an action, then it goes in the Body section of the `FOR` loop.

Example 2.5 – Counter-Controlled Loop

Demo	This example will demonstrate how to design a counter-controlled loop.
Problem	Sue heard the following story in class one day: When Carl Friedrich Gauss was 6, his schoolmaster, who wanted some peace and quiet, asked the class to add up the numbers 1 to 100. “Class,” he said, coughing slightly, “I’m going to ask you to perform a prodigious feat of arithmetic. I’d like you all to add up all the numbers from 1 to 100, without making any errors.” “You!” he shouted, pointing at little Gauss, “How would you estimate your chances of succeeding at this task?” “Fifty-fifty, sir,” stammered little Gauss, “no more...” What the schoolmaster did not realize was that young Gauss figured out that adding the numbers from 1 to n is the same as: $sum = (n + 1)n / 2$. Sue wants to write a program to verify this. Her program will both add the numbers one by one, and use Gauss’ equation.
Solution	With counter-controlled loops, four questions need to be answered: • Initialization: The loop starts at 1: <code>int count = 1</code> • End: The loop stops after the number is reached: <code>count <= n</code> • Update: The loop counts by 1’s: <code>count++</code> • Body: Every iteration we increase the sum by count: <code>sum += count</code> With these four answers, we can write the function. <pre>int computeLoop(int n) { int sum = 0; for (int count = 1; count <= n; count++) sum += count; return sum; }</pre>
Challenge	As a challenge, find values where Gauss’s equation does not work. Can you modify the program so it works with all integer values?
See Also	The complete solution is available at 2-5-counterControlled.cpp or: <pre>/home/cs124/examples/2-5-counterControlled.cpp</pre> 

Event-Controlled

An event-controlled loop is a loop that continues until a given event occurs. The number of repetitions is typically not known before the program starts. For example, if I were to write a program to prompt the user for her age but would re-prompt if the age was negative, an event-controlled loop is probably the right tool for the job. Typically event-controlled loops use `while` or `do-while` statements, depending if the loop needs to execute at least once.

Boolean expression:

How do you know when you are done? Event controlled loops keep going until an event has occurred. In this case, until the GPA is within an acceptable range.

```
while (gpa > 4.0 || gpa < 0.0)
{
    cout << "Enter your GPA: ";
    cin >> gpa;
}
```

Body:

What changes every iteration? Typically something happens during event-controlled loops. This may be a re-prompt or a value is updated some other way. The programmer cannot predict how many iterations will be required; it depends on execution.

When designing with an event-controlled loop, it is usually helpful to answer the following two questions:

- **How do you know when you are done?** In other words, what is the termination condition? This condition maps directly to the controlling Boolean expression of a `WHILE` or `DO-WHILE` loop.
- **What changes every iteration?** Unlike counter-controlled loops where the counter changes in a predictable way, event controlled loops are typically driven by an external event. Usually there needs to be code in an event-controlled loop to re-query this external event. Often this comes in the form of re-prompting the user for input or reading more data from a file.

Sam's Corner



Notice how the two parts to an event-controlled loop (ending condition and what changes every iteration) look a lot like the two of the four parts of a counter-controlled loop (start, ending condition, counting, and what happens every iteration). This is because an event-controlled loop is a sub-set of a counter-controlled loop. If the counter-controlled loop does not have a starting condition or a counting component, then it probably is an event-controlled loop.

All `FOR` loops can be converted into a `WHILE` loop.

```
for ( i = 0;    // initialize
      i < 10;   // condition
      i++)     // increment
    cout << i << endl;
```

```
i = 0;          // initialize
while (i < 10)   // condition
{
    cout << i << endl;
    i++;        // increment
}
```

All `WHILE` loops can be converted into a `FOR` loop.

```
while (grade < 70.0)
    grade = takeClassAgain();
```

```
for (; grade < 70.0; )
    grade = takeClassAgain();
```

Example 2.5 – Event-Controlled Loop

Demo

This example will demonstrate how to design an event-controlled loop.

Problem

Write a program to keep prompting the user for his GPA until a valid number is entered.

```
Please enter your GPA (0.0 <= gpa <= 4.0): 4.1
Please enter your GPA (0.0 <= gpa <= 4.0): -0.1
Please enter your GPA (0.0 <= gpa <= 4.0): 3.9
GPA: 3.9
```

Solution

With event-controlled loops, two questions need to be answered:

- **End condition:** The loop ends when the condition is reached $0 \leq \text{gpa} \leq 4.0$.
- **Update:** Every iteration, we re-prompt the user for the GPA.

With these two answers, we can write our pseudocode:

```
WHILE gpa > 4.0 or gpa < 0.0
  PROMPT for gpa
  GET gpa
```

With this pseudocode, it is straight-forward to write the function:

```
float getGPA()
{
    float gpa = -1.0; // any value outside the expected range will do

    // loop until a valid value is received
    while (gpa > 4.0 || gpa < 0.0)
    {
        cout << "Please enter your GPA (0.0 <= gpa <= 4.0): ";
        cin >> gpa;
    }

    // return with the loot
    assert(gpa <= 4.0 && gpa >= 0.0); // paranoia will destroy-ya
    return gpa;
}
```

Observe how much more complex the C++ for `getGPA()` is than the pseudocode. This is because variable initialization, comments, the text of the prompt, and asserts are not required for pseudocode

Challenge

As a challenge, try to change the above function from a WHILE loop to a DO-WHILE loop. Both loops are commonly event-controlled.

See Also

The complete solution is available at [2-5-eventControlled.cpp](#) or:

```
/home/cs124/examples/2-5-eventControlled.cpp
```



Sentinel-Controlled

A sentinel is a guardian or gatekeeper. In the case of a sentinel-controlled loop, the sentinel is a variable used to determine if a loop is to continue executing or if it is to terminate. We typically use a sentinel when multiple events could cause the loop to terminate. In these cases, the sentinel could be changed by any of the events. Typically event-controlled loops use `while` or `do-while` statements where the sentinel is a `bool` used as the condition. Perhaps, this is best explained by example.

Example 2.5 – Sentinel-Controlled Loop

Demo

This example will demonstrate how to design a sentinel-controlled loop.

Problem

Consider a professor trying to determine if a student has passed his class. There are many criteria to be taken into account (the grade and whether he cheated, to name a few). Rather than making a single highly-complex controlling Boolean expression, he decides to use a sentinel-controlled loop:

```
Welcome to CS 124!
What is your class grade? 109
Did you cheat in the class? (y/n) y

Welcome to CS 124!
What is your class grade? 81
Did you cheat in the class? (y/n) n
Great job! Get ready for CS 165
```

Solution

The solution is to have one variable (passed) be set by a variety of conditions.

```
{
    bool passed = false;                // the sentinel. Initially we have
                                        //      not passed the class

    // the main loop
    while (!passed)                     // common sentinel, read
    {                                   //      "while not passed..."

        cout << "\nWelcome to CS 124!\n";

        // if you got a C or better, you may have passed...
        float grade;
        cout << "What is your class grade? ";
        cin >> grade;
        if (grade >= 60.0)
            passed = true;               // one of the ways the
                                        //      sentinel may change

        // if you cheated, you did not pass
        char cheated;
        cout << "Did you cheat in the class? (y/n) ";
        cin >> cheated;
        if (cheated == 'y' || cheated == 'Y')
            passed = false;              // another sentinel condition
    }
    cout << "Great job! Get ready for CS 165!\n";
}
```

See Also

The complete solution is available at [2-5-sentinelControlled.cpp](#) or:

```
/home/cs124/examples/2-5-sentinelControlled.cpp
```



Problem 1

Write a program to put the alphabet on the screen. A.K.A. Sesame Street Karaoke.

A
B
C
...
Y
Z

Answer:

Please see page 182 for a hint.

Problem 2

Write a program to display the multiplication table of numbers less than 6.

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25

Answer:

Please see page 182 for a hint.

Problem 3

Write a program to compute how many numbers under a user-specified value are both odd and a multiple of 5.

What is the number: 20

The number of values under 20 that are both odd and a multiple of 5 are: 2

Answer:

Please see page 182 for a hint.

Problem 4

Write a function to compute whether a number is prime:

```
bool isPrime(int number);
```

Answer:

Please see page 49 for a hint.

Assignment 2.5

Write a function (`displayTable()`) to display a calendar on the screen. The function will take two parameters:

- `numDays`: The number of days in a month.
- `offset`: The offset from Monday. If the offset is zero, then the month starts on Monday. If the offset is 2, the month starts on Wednesday. If the offset is 6, the month starts on Sunday.

This function will be “very similar” to the `displayTable()` function in Project 2. Please see the project for details on the spacing between the columns and a hint on how the algorithm might work. Next write `main()` so that it prompts the user for the number of days in the month and the offset.

Note that this is probably the most difficult assignment of the semester. Of course, this will also get you very far along on the project as well. Please, allocate a couple hours for this assignment.

Example

Three examples. The user input is underlined.

Number of days: 30

Offset: 3

Su	Mo	Tu	We	Th	Fr	Sa
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	

Number of days: 28

Offset: 0

Su	Mo	Tu	We	Th	Fr	Sa
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28						

Number of days: 31

Offset: 6

Su	Mo	Tu	We	Th	Fr	Sa
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

Assignment

The test bed is available at:

```
testBed cs124/assign25 assignment25.cpp
```

Don't forget to submit your assignment with the name “Assignment 25” in the header,

Please see page 182 for a hint.